Computer Vision Homework 4

Name: NICKHOLAS

Student ID: M11515801

Department: NTUST CSIE

Write a program to do the following requirement.

I am using python 3.10 to implement all the binary morphology requirements.

Using OpenCV (cv2) to read and save images, and the processing is done on NumPy arrays.

Below are the settings used for every part before jumping into each requirement:

(a) lena.bmp is binarized with threshold 128 and then convert to 0/1 for the morphology operations.

```python
# Binarization (from earlier homework): returns 0 / 255
def binarize(img):
    h, w = img.shape
    binary = np.zeros((h, w), dtype=np.uint8)

    for i in range(h):
        for j in range(w):
            binary[i][j] = 255 if img[i][j] >= 128 else 0

    return binary
```

```python
# binarize image (0 / 255), then convert to 0 / 1 for the morphology functions
binary_255 = binarize(gray)
binary = (binary_255 // 255).astype(np.uint8)
```

(b) Using octagonal 3-5-5-5-3 kernel.

```python
# Octagonal 3-5-5-5-3 kernel, origin at the center
OCTAGON = [(dy, dx)
           for dy, half in [(-2, 1), (-1, 2), (0, 2), (1, 2), (2, 1)]
           for dx in range(-half, half + 1)]
```

(c) A helper function shift is used in the dilation, erosion and hit-and-miss functions (opening and closing are built from dilation and erosion). It moves the whole image by (dy, dx), so that every pixel can look at its neighbor at one kernel position at the same time. Pixels that fall outside the image are filled with 0 (black), so the outside of the image is treated as background.

```python
def shift(img, dy, dx):
    h, w = img.shape
    out = np.zeros_like(img)
    ys, ye = max(0, -dy), min(h, h - dy)
    xs, xe = max(0, -dx), min(w, w - dx)
    out[ys:ye, xs:xe] = img[ys + dy:ye + dy, xs + dx:xe + dx]
    return out
```

Below are the results and explanations

(a) Dilation



Explanation:

```
# (a) Dilation
def dilation(img, kernel):
    out = np.zeros_like(img)
    for dy, dx in kernel:
        out |= shift(img, -dy, -dx)
    return out
```

The dilation function is defined. If at least one pixel within the 3-5-5-5-3 kernel range is not 0, the center pixel is assigned to 255, so the white regions expand.

(b) Erosion



Explanation:

```
# (b) Erosion
def erosion(img, kernel):
    out = np.ones_like(img)
    for dy, dx in kernel:
        out &= shift(img, dy, dx)
    return out
```

For the erosion function, if all pixels within the 3-5-5-5-3 kernel range are not 0, the center pixel is assigned to 255, otherwise it is assigned to 0. As a result, the white regions shrink and thin white lines and small white spots are removed.

(c) Opening



Explanation:

```python
# (c) Opening
def opening(img, kernel):
    return dilation(erosion(img, kernel), kernel)
```

Opening runs erosion first, then dilation with the same kernel. The erosion removes small white spots and thin lines, then the dilation grows the remaining white regions back to roughly their original size. The removed parts cannot come back, so the result keeps the large white shapes without the small white details.

(d) Closing



Explanation:

```python
# (d) Closing
def closing(img, kernel):
    return erosion(dilation(img, kernel), kernel)
```

Closing runs dilation first, then erosion with the same kernel. The dilation fills small black holes and narrow gaps, then the erosion shrinks the white regions back to roughly their original size. The filled parts stay filled, so the result keeps the overall shape of the white regions without the small black details.

(e) Hit-and-miss transform



Explanation:

```
# "L" shaped kernels for the upper-right corner
J_KERNEL = [(0, 0), (0, -1), (1, 0)]
K_KERNEL = [(-1, 0), (-1, 1), (0, 1)]
```

```
# (e) Hit-and-miss transform
def hit_and_miss(img, j_kernel, k_kernel):
    return erosion(img, j_kernel) & erosion(1 - img, k_kernel)
```

The hit-and-miss transform finds the pixels whose neighbors match an exact pattern. Here the “L” shaped kernel is used to detect the upper-right corners of the white regions. It uses two kernels: J, which must be white (the center, left and below pixels), and K, which must be black (the above, above right and right pixels). The function erodes the image with J, erodes the complement of the image (white and black swapped) with K, and keeps only the pixels that pass both, using AND, a logical operation that is true only when both inputs are true. 1 means white (true) and 0 means black (false). As a result, only the upper-right corner pixels of the white regions are marked.